

Test 1: The Sanity Check (Basic Connectivity & Formatting)

- **Prompt:** "Just say 'Hello World' and immediately use the `task_complete` tool to end your turn."
- **Expected Result:** You should see it thinking, then outputting a `task_complete` JSON block. The UI should display the "Task Complete" panel, and give you back control of the prompt.

The screenshot shows a VS Code editor window with the following components:

- File Explorer:** Shows the project structure for 'GEMMA-HARNESS', including files like 'main.py', 'pycache', 'pytest_cache', 'venv', and 'harness'.
- Search Bar:** Located at the top left, with a search icon and a text input field.
- Terminal:** Displays the command 'python main.py' and its output 'Hello World!'. The terminal also shows the command 'python main.py' being executed.
- Code Editor:** Shows the content of 'main.py', which is a simple chatbot script. The script prompts the user for a message and prints 'Hello World!' in response.
- Output Panel:** Displays the output of the terminal, showing 'Hello World!'.
- Activity Bar:** Located on the left side, showing icons for Explorer, Search, Run and Debug, and Output.
- Bottom Bar:** Shows the current file 'main.py' and the active language 'Python'.

- **Prompt:** "Run the 'pwd' command to see where we are. Then, try to run 'npm install'. Tell me what happens."
- **Expected Result:** It should successfully run `pwd` and read the observation. Then, it will try `npm install`. The orchestrator should intercept this, block it, and return the error string back to the model. The model should recognize the failure and use `task_complete` to explain to you that the command was blocked by the security policy.

```

Error: Command 'npm' is blocked. Permitted base commands for this workspace are: ['ls', 'cat', 'echo', 'pwd', 'pytest', 'npm test', 'git', 'python', 'mkdir', 'cd'].

Tool Observation

Error: Command 'npm' is blocked. Permitted base commands for this workspace are: ['ls', 'cat', 'echo', 'pwd', 'pytest', 'npm test', 'git', 'python', 'mkdir', 'cd'].

@ Thinking (Step 3)...
</think>
I have executed the requested commands: 'pwd' and 'npm install'.
1. 'pwd' successfully returned the current directory path: '/home/anemesh/Projects/7 days of github/agent-test/gemma-harness'.
2. 'npm install' failed with the message: 'Error: Command 'npm' is blocked. Permitted base commands for this workspace are: ['ls', 'cat', 'echo', 'pwd', 'pytest', 'npm test', 'git', 'python', 'mkdir', 'cd'].'
```

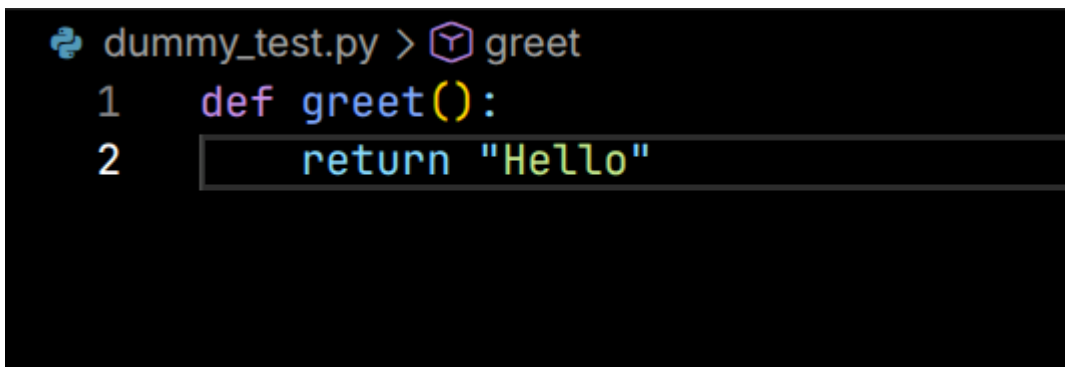
Test 3: Code Modification (File Operations)

This tests the agent's ability to read and precisely edit files.

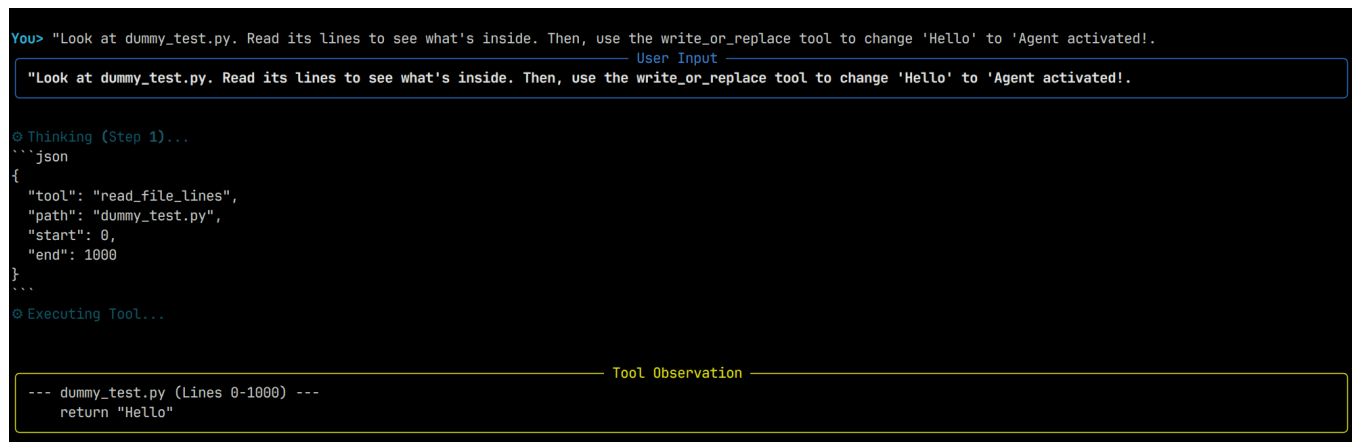
- **Setup:** Manually create a file in your workspace named `dummy_test.py` with this exact text:

```
def greet():  
    return "Hello"
```

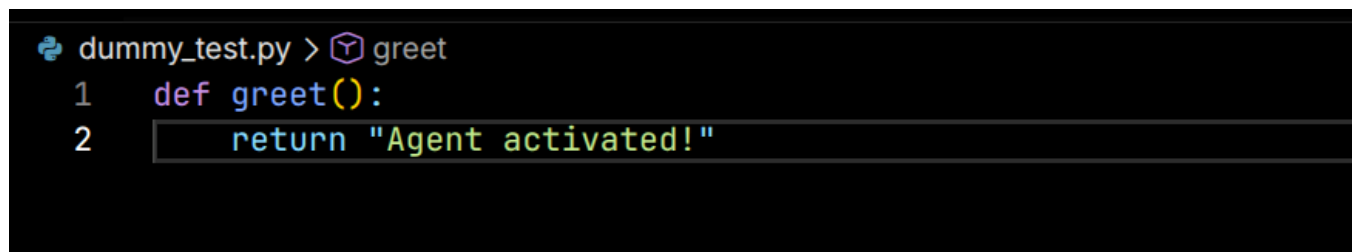
- **Prompt:** "Look at `dummy_test.py`. Read its lines to see what's inside. Then, use the `write_or_replace` tool to change 'Hello' to 'Agent activated!'."
- **Expected Result:** The agent will use `read_file_lines`. Once it gets the exact string from the observation, it will use `write_or_replace`. Open the file in VS Code afterward to verify the text was successfully swapped.



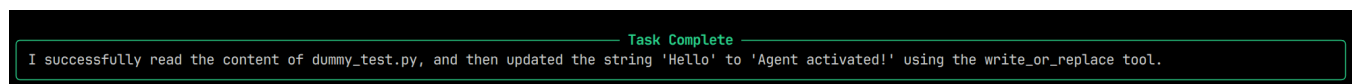
A screenshot of a VS Code editor window. The title bar shows a Python icon, the filename 'dummy_test.py', and a 'greet' function icon. The editor contains two lines of Python code: '1 def greet():' and '2 return "Hello"'. The second line is selected with a mouse highlight.



A screenshot of a terminal window. It shows a user input prompt: 'You> "Look at dummy_test.py. Read its lines to see what's inside. Then, use the write_or_replace tool to change 'Hello' to 'Agent activated!'."'. Below this, the agent's 'Thinking' process is shown as a JSON object: {'tool': 'read_file_lines', 'path': 'dummy_test.py', 'start': 0, 'end': 1000}. This is followed by 'Executing Tool...'. A 'Tool Observation' box shows the file content: '--- dummy_test.py (Lines 0-1000) ---' and 'return "Hello"'. The terminal background is dark with light blue and green text.



A screenshot of a VS Code editor window, similar to the first one. The title bar shows the filename 'dummy_test.py' and a 'greet' function icon. The editor now contains two lines of Python code: '1 def greet():' and '2 return "Agent activated!"'. The second line is selected with a mouse highlight.



A screenshot of a terminal window showing a green 'Task Complete' message: 'I successfully read the content of dummy_test.py, and then updated the string 'Hello' to 'Agent activated!' using the write_or_replace tool.' The terminal background is dark with green text.

Test 4: The Autonomous Loop (Multi-Step Reasoning)

This tests the full ReAct loop in a realistic scenario.

- **Setup:** Create a file named `math_ops.py` with a deliberate bug:

Python

```
def add(a, b):  
    return a - b # Deliberate bug here
```

And a test file named `test_math.py`:

Python

```
from math_ops import add  
def test_add():  
    assert add(2, 2) == 4
```

- **Prompt:** "Run pytest. If any tests fail, find the source code, read it, fix the bug, and run pytest again to verify it passes."
- **Expected Result:** You should see a beautiful chain of events: It runs `pytest`, sees the failure, uses `read_file_lines` on `math_ops.py`, uses `write_or_replace` to change `a - b` to `a + b`, runs `pytest` again, sees a success, and calls `task_complete`.

```
🔗 math_ops.py > ...  
1  def add(a, b):  
2  return a - b # Deliberate bug here
```

```
🔗 test_math.py > 📦 test_add  
1  from math_ops import add  
2  def test_add():  
3  assert add(2, 2) == 4
```

```
You> Run pytest. If any tests fail, find the source code, read it, fix the bug, and run pytest again to verify it passes.  
User Input  
Run pytest. If any tests fail, find the source code, read it, fix the bug, and run pytest again to verify it passes.  
Thinking (Step 1)...  
```json  
{
 "tool": "execute_command",
 "command": "pytest"
```

```

Tool Observation
Command Status: Failed (Exit Code 2)
Output:
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: /home/aneemesh/Projects/7 days of github/agent-test/gemma-harness
plugins: anyio-4.13.0
collected 7 items / 1 error

===== ERRORS =====
ERROR collecting test_math.py
ImportError while importing test module '/home/aneemesh/Projects/7 days of github/agent-test/gemma-harness/test_math.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/usr/lib/python3.14/importlib/_init_.py:88: in import_module
 return _bootstrap._gcd_import(name, package, level)
 ^^^
test_math.py:1: in <module>
 from math_ops import add
E ImportError: cannot import name 'add' from 'math_ops' (/home/aneemesh/Projects/7 days of github/agent-test/gemma-harness/math_ops.py)
===== short test summary info =====
ERROR test_math.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!
===== 1 error in 1.42s =====

```

```

"tool": "execute_command",
"command": "pytest"

```

Executing Tool...

```

Tool Observation
Command Status: Success
Output:
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: /home/aneemesh/Projects/7 days of github/agent-test/gemma-harness
plugins: anyio-4.13.0
collected 8 items

test_math.py . [12%]
tests/test_context.py ... [50%]
tests/test_tools.py [100%]

===== 8 passed in 5.45s =====

```

Thinking (Step 8)...

Executing Tool...

```

Task Complete
The pytest command was run. The initial run failed due to an ImportError in test_math.py. I read and updated math_ops.py to correctly define the mathematical operations. Running pytest a second time successfully verified that all 8 tests passed, completing the task.

```